

**Name :- Shrivardhan Bharat Kate**

**Roll No.: 198**

**Subject :- Python Programming**

**Practical 8 - Programs based on OOP concepts (inheritance, polymorphism, encapsulation).**

## A. Single Inheritance

Create a class Person with attributes name and age. Derive a class Student that adds marks and displays all details.

### Program Code:-

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, marks):
        super().__init__(name, age)
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

s = Student("Shrivardhan", 21, 85)
s.display()
```

### Output:-

```
===== RESTART: C:/Users/Student/Documents/Practical8.py :
Name: Shrivardhan
Age: 21
Marks: 85
>>>|
```

## B. Multilevel Inheritance

Create a class Animal with method eat().

Derive class Dog with method bark(), and further derive class Puppy with method weep().

**Program code:-**

```
class Animal:
    def eat(self):
        print("Animal eats")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

class Puppy(Dog):
    def weep(self):
        print("Puppy weeps")

p = Puppy()
p.eat()
p.bark()
p.weep()
```

**Output:-**

```
===== RESTART: C:/Users/Student/Documents/Practical8.py =====
Animal eats
Dog barks
Puppy weeps
>>>
```

### C. Hierarchical Inheritance

Create a base class Shape.

Derive classes Rectangle and Triangle with methods to calculate area.

#### Program Code:-

```
class Shape:
    pass

class Rectangle(Shape):
    def area(self, l, b):
        return l * b

class Triangle(Shape):
    def area(self, b, h):
        return 0.5 * b * h

r = Rectangle()
t = Triangle()

print("Rectangle Area:", r.area(6, 5))
print("Triangle Area:", t.area(6, 5))
```

#### Output:-

```
===== RESTART: C:/Users/Student/Documents/Practical8.py =====
Rectangle Area: 30
Triangle Area: 15.0
>>> |
```

## D. Method Overriding (Polymorphism)

Create a class Employee with method work(). Override the method in subclasses Manager and Developer.

### Program Code:-

```
class Employee:
    def work(self):
        print("Employee works")

class Manager(Employee):
    def work(self):
        print("Manager manages the team")

class Developer(Employee):
    def work(self):
        print("Developer writes the code")

e1 = Manager()
e2 = Developer()

e1.work()
e2.work()
```

### Output:-

```
===== RESTART: C:/Users/Student/Documents/Practical8.
Manager manages the team
Developer writes the code
>>>
```

## E. Polymorphism Using Same Method Name

Create classes Bird and Airplane with a method fly() that shows different behavior.

### Program Code:-

```
class Bird:
    def fly(self):
        print("Bird flies in the sky")

class Airplane:
    def fly(self):
        print("Airplane also flies but with engines")

objects = [Bird(), Airplane()]

for obj in objects:
    obj.fly()
```

### Output:-

```
===== RESTART: C:/Users/Student/Documents/Practical8.
Bird flies in the sky
Airplane also flies but with engines
>>>
```

## F. Encapsulation with Private Variable

Create a class BankAccount with a private variable balance.

Add methods to deposit, withdraw, and display balance.

**Program Code:-**

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited: {amount}")
        else:
            print("Invalid deposit amount")

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrawn: {amount}")
        else:
            print("Insufficient balance")

    def display_balance(self):
        print(f"Current Balance: {self.__balance}")

acc = BankAccount(1000)
acc.deposit(500)
acc.withdraw(200)
acc.display_balance()
```

### Output:-

```
=== RESTART: C:/Users/NIWAS/AppData/Local/Programs/Python/Python312/niwass.py ==  
Deposited: 500  
Withdrawn: 200  
Current Balance: 1300  
>>> |
```



## G. Encapsulation with Getter and Setter

Create a class Employee with private variable salary. Use getter and setter methods to access and modify salary.

### Program Code:-

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
        else:
            print("Invalid salary")

emp = Employee(25000)

print("Initial Salary:", emp.get_salary())

emp.set_salary(40000)
print("Updated Salary:", emp.get_salary())
```

### Output:-

```
=== RESTART: C:/Users/NIWAS/AppData/Local/Programs/Python/Python312,
Initial Salary: 25000
Updated Salary: 40000
>>>
```

## H. Hybrid

Create a class Vehicle with method start(). Derive classes Car and Bike.

Override the start() method in both classes.

### Program Code:-

```
class Vehicle:
    def start(self):
        print("Vehicle is starting...")

class Car(Vehicle):
    def start(self):
        print("Car starts with key ignition")

class Bike(Vehicle):
    def start(self):
        print("Bike starts with self/kick")

v = Vehicle()
c = Car()
b = Bike()

v.start()
c.start()
b.start()
```

### Output:-

```
=== RESTART: C:/Users/NIWAS/AppData/Local/Programs/Python/Python312
Vehicle is starting...
Car starts with key ignition
Bike starts with self/kick
>>> |
```